

01 PROJECT OVERVIEW

The goal of this project is to build a **Lost & Found web platform** where users can report lost items or post items they have found. Other users can browse the platform and contact the owner to return the item.

This project will help new developers practice **full-stack development using the MERN stack**, including frontend UI development, backend APIs, database integration, and image uploads.

Learning Objectives

- Understand and apply MERN stack architecture end-to-end
- Build and consume REST APIs with Express.js
- Structure React components and manage application state
- Model data effectively with MongoDB and Mongoose
- Handle file/image uploads in a full-stack environment
- Follow a professional full-stack project workflow

02 TECH STACK

Layer	Technology	Purpose
Frontend	React.js	UI component library
Frontend	Axios	HTTP requests to backend
Frontend	Tailwind CSS / CSS	Styling and layout
Backend	Node.js	JavaScript runtime
Backend	Express.js	REST API framework
Database	MongoDB	NoSQL document store
Database	Mongoose	ODM for MongoDB
Storage	Cloudinary / Local	Image upload handling
Auth (opt.)	JWT	User authentication

03 CORE FEATURES

1. Post Lost Item

Users can report items they have lost.

Required Fields:

- Item Name
- Description
- Location Lost
- Date Lost

- Contact Info (name, email, phone)
- Image Upload

2. Post Found Item

Users can report items they have found.

Required Fields:

- Item Name
- Description
- Location Found
- Date Found
- Contact Info (name, email, phone)
- Image Upload

3. Browse Items

Users can browse all posts on the platform, filtered by type.

Required Fields:

- Lost Items feed
- Found Items feed
- Card/list layout with preview images

4. Item Details Page

Each item has its own dedicated detail page.

Required Fields:

- Full item image
- Description and location
- Date reported
- Full contact information

5. Contact Owner

Users can reach the poster directly via the details they provided.

Required Fields:

- Email link
- Phone number display

04 REQUIRED PAGES

Page	Description
Home Page	Displays the latest lost and found items as card previews.

Report Lost Item	Form page where users submit details for a lost item.
Report Found Item	Form page where users submit details for a found item.
Browse Items	Displays all items in a list or card layout with Lost/Found filters.
Item Details	Full information view for a selected item, including contact info.
Admin / Manage (Optional)	Admin dashboard to edit or delete any post.

05 PROJECT FOLDER STRUCTURE

Follow this suggested structure for a clean, maintainable project:

```
lost-found-app/  
  client/ (React Frontend)  
    src/  
      components/  
        ItemCard.jsx  
        Navbar.jsx  
        Footer.jsx  
      pages/  
        Home.jsx  
        ReportLost.jsx  
        ReportFound.jsx  
        BrowseItems.jsx  
        ItemDetails.jsx  
      services/  
        api.js  
        App.jsx  
      server/ (Node.js Backend)  
        models/  
          Item.js  
        routes/  
          itemRoutes.js  
        controllers/  
          itemController.js  
        uploads/ (local image storage)  
        server.js
```

06 MONGODB DATA SCHEMA

Item Model — *server/models/Item.js*

Field	Type	Description
-------	------	-------------

title	String	Name of the lost or found item
description	String	Detailed description of the item
type	"lost" "found"	Whether the item was lost or found
location	String	Where the item was lost or found
date	Date	Date of the incident
image	String	Path or URL of the uploaded image
contactName	String	Full name of the person who posted
contactEmail	String	Email address for contact
contactPhone	String	Phone number for contact
createdAt	Date	Timestamp when the post was created

07 WEEKLY TIMELINE

Day 1	Project setup, environment configuration, and database schema design
Day 2	Backend API development — CRUD routes for items using Express + Mongoose
Day 3	React project setup, routing, and page scaffolding
Day 4	Form implementation and full API integration (Axios calls)
Day 5	Image upload feature using Cloudinary or local Multer middleware
Day 6	UI polish, responsive design improvements, and testing
Day 7	Final deployment, README writing, and submission

08 SUBMISSION REQUIREMENTS

Each team must submit the following before the deadline:

- A **working MERN application** (frontend + backend running together)
- A **GitHub repository** with clean commit history
- A **clean folder structure** as outlined in Section 05
- A **basic, functional UI design** (does not need to be pixel-perfect)
- A **README.md** file explaining how to install and run the project locally

09 NOTES & TIPS

- Commit your code frequently — at least once per day per team member.
- Use branches (e.g. *feature/report-lost*) to avoid conflicts.
- Test your API routes with Postman or Thunder Client before connecting the frontend.
- Keep your `.env` file out of Git — use `.gitignore` from day one.

- Ask questions early — do not wait until Day 6 to ask for help.
- JWT authentication is optional but encouraged for bonus marks.

Good luck to all teams. Focus on clean code, good communication, and steady daily progress.